Assignment 6: Visitor Pattern

File System Visitor | Java 17

Astana IT University · School of Computer Engineering
ShP-2216 – Software Design Patterns (OP 6B06102)

1. Introduction

The project implements Option A, the File System Visitor. Its object structure is a small in-memory hierarchy containing File and Directory elements. A directory stores an ordered collection of files and subdirectories. The same structure is processed by three independent operations: total byte size, predicate-based search, and file counting.

Visitor is suitable because the element types are stable while operations can grow. File and Directory expose accept(FileSystemVisitor); the visitor interface declares visitFile(File) and visitDirectory(Directory). Adding an operation requires a new visitor rather than a new operation-specific method on each element.

1.1 Implemented operations

Concrete visitor	Operation
SizeCalculatorVisitor	Recursively sums file sizes using long and checked addition.
SearchVisitor	Collects files satisfying a Predicate<File> in depth-first order.
FileCountVisitor	Counts file entries; demonstrates a third operation added without element edits.

1.2 Shared demonstration structure

```
project/
  readme.txt        100 bytes
  docs/
    notes.TXT       200 bytes
    guide.pdf       300 bytes
  images/
    logo.png       5000 bytes
  todo.txt          500 bytes
```

Expected total: 100 + 200 + 300 + 5000 + 500 = 6100 bytes. There are five files and three case-insensitive .txt matches. Directory metadata is not counted.

2. UML class diagram

Figure 1 shows the element hierarchy, visitor interface, concrete operations, and child collection. Dashed hollow-triangle arrows denote interface realization. The open diamond denotes aggregation: a directory contains zero or more child elements, and an immutable child can be shared.

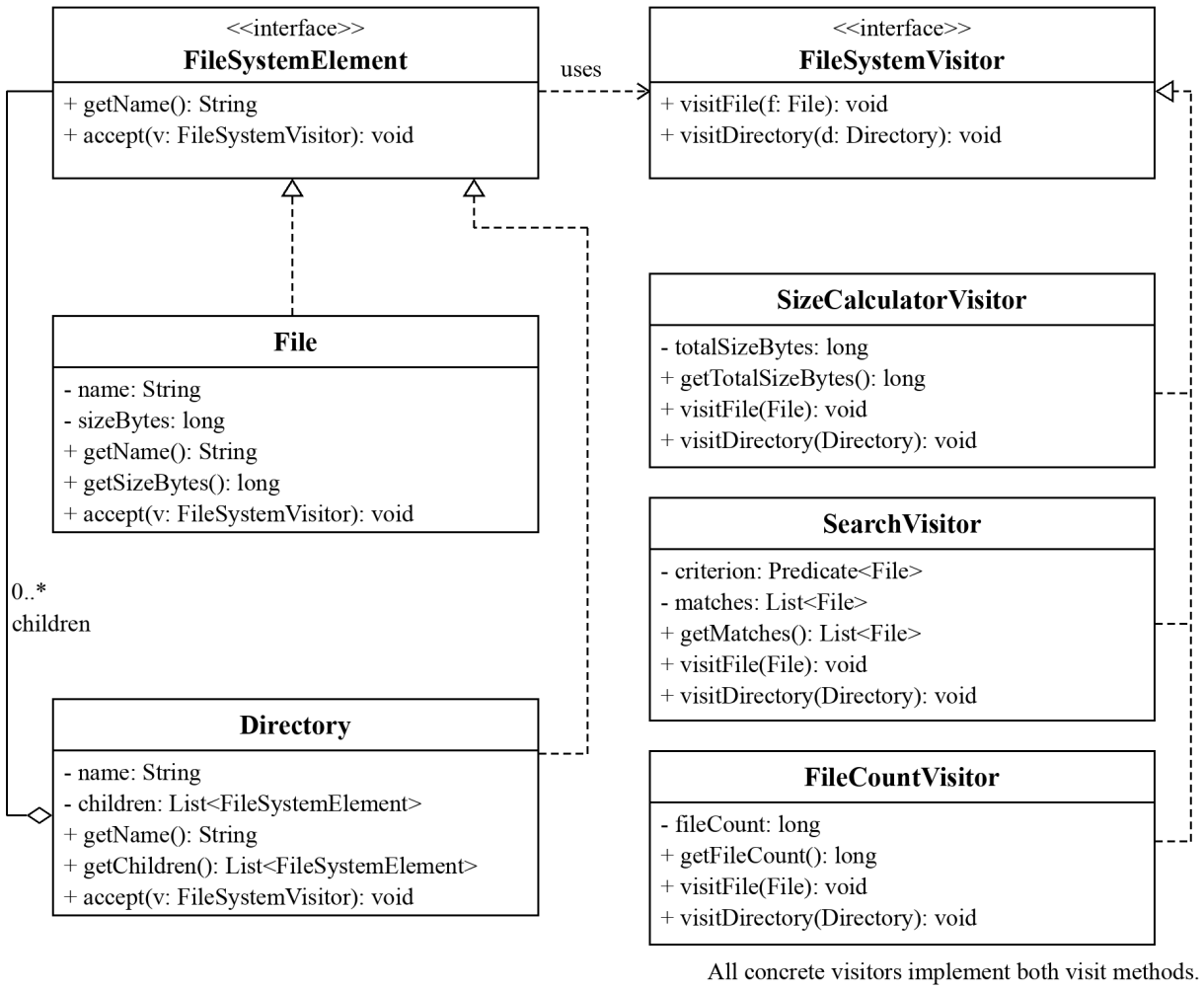


Figure 1. Class relationships for the implemented File System Visitor.

2.1 Responsibilities and traversal

File stores a name and size; Directory stores a name and an immutable child list. Their accept methods only dispatch. The client starts traversal with root.accept(visitor). Each concrete visitor implements both visit methods and recursively calls child.accept(this) from visitDirectory, preserving child order. All visitors run on the same root.

Editable diagram source is included as docs/uml.puml; SVG and PNG versions are also supplied. The client Demo constructs the structure and presents results, so it is omitted from the pattern-role diagram.

3. Double dispatch

Double dispatch selects behavior using both the concrete element type and the concrete visitor type. The implementation uses two ordinary Java virtual calls; it does not dynamically select an overload from a parameter's runtime type.

```
FileSystemElement element = new File("readme.txt", 100);
FileSystemVisitor visitor = new SizeCalculatorVisitor();
element.accept(visitor);
```

First, the runtime type of element selects File.accept. Inside that method, this has compile-time type File, which fixes the visitFile(File) signature:

```
@Override
public void accept(FileSystemVisitor visitor) {
    visitor.visitFile(this);
}
```

Second, the runtime type of visitor selects SizeCalculatorVisitor.visitFile. With a SearchVisitor instance, the same File.accept call selects SearchVisitor.visitFile instead. Directory follows the corresponding visitDirectory route. Its visitor handler continues through the children, repeating the two calls for each element.

4. Clean Code principles

4.1 Meaningful names and explicit units

```
private final long sizeBytes;          // element data
private long totalSizeBytes;           // visitor result
public long getTotalSizeBytes() { return totalSizeBytes; }
```

Names distinguish stored element state from an accumulated operation result. The Bytes suffix removes unit ambiguity, while visitFile and visitDirectory state exactly which element a handler processes. This reduces interpretation and makes the API easier to defend.

4.2 Polymorphism replaces type-check chains

```
// File.accept: the concrete file selects its handler.
visitor.visitFile(this);
// Directory.accept: the concrete directory selects its handler.
visitor.visitDirectory(this);
```

Each element dispatches directly to its matching method. Production code needs no instanceof, reflection, cast, or type-switch chain. New operations implement the visitor interface and reuse this dispatch unchanged, keeping the element hierarchy free of operation selection logic.

4. Clean Code principles (continued)

4.3 Single Responsibility Principle

```
// SizeCalculatorVisitor.visitFile:
totalSizeBytes = Math.addExact(totalSizeBytes, file.getSizeBytes());
// SearchVisitor.visitFile:
if (criterion.test(file)) {
    matches.add(file);
}
```

Each visitor implements one operation across both element types: size, search, or count. Elements store validated data; Demo handles presentation. Calculating bytes does not print output or search, so an operation can change without mixing unrelated responsibilities.

4.4 Encapsulation and immutable boundaries

```
// Directory constructor and getter:
this.children = List.copyOf(children);
public List<FileSystemElement> getChildren() { return children; }
// SearchVisitor result:
return List.copyOf(matches);
```

The constructor snapshots the caller's list and rejects null lists or entries. The returned child list cannot be modified; the supplied File and Directory classes are final with immutable fields. Search returns a separate unmodifiable snapshot, so callers cannot change the visitor's internal result list. Visitors see only public read-only accessors, not writable fields.

4.5 Open/Closed Principle and demonstrated extensibility

```
public final class FileCountVisitor implements FileSystemVisitor {
    private long fileCount;
    public void visitFile(File file) {
        fileCount = Math.incrementExact(fileCount);
    }
    public void visitDirectory(Directory directory) {
        for (FileSystemElement child : directory.getChildren()) {
            child.accept(this);
        }
    }
    public long getFileCount() { return fileCount; }
}
```

FileCountVisitor was added after size and search in commit 682e2b2. That commit adds the operation and demo without modifying File, Directory, or FileSystemVisitor. This is an executable extension example. The hierarchy is open to new operations; adding a new element type would require changing the visitor contract and all implementations.

5. Actual sample output and verification

The following output was captured from ./run.sh. Size, search, and count each receive the same root instance; each operation uses a fresh visitor.

```
File System Visitor Demo
Structure: project/
  readme.txt (100 bytes)
  docs/
    notes.TXT (200 bytes)
    guide.pdf (300 bytes)
  images/
    logo.png (5000 bytes)
  todo.txt (500 bytes)

Total size: 6100 bytes
Text files (.txt, case-insensitive):
  readme.txt (100 bytes)
  notes.TXT (200 bytes)
  todo.txt (500 bytes)
File count: 5
```

5.1 Build and automated checks

```
./run.sh
./test.sh
# Final test summary:
All 30 tests passed.
```

Both scripts compile with javac --release 17 -Xlint:all -Werror. The project uses only the Java standard library and a dependency-free test runner. Tests throw AssertionError on failure even without the -ea JVM option.

Checked behavior	**Evidence**
Visitor correctness	Nested, empty, and direct-file operations; explicit dispatch test; all visitors share one structure.
Search behavior	Case-insensitive extension and size predicates; no matches; directories excluded; immutable snapshots.
Data boundaries	Nulls, blank names, negative sizes, defensive child copy, and unmodifiable child collection.
Numerical safety	Totals above Integer.MAX_VALUE and explicit ArithmeticException on long overflow.
Integration	Demo output includes the actual total, matching file names and file count.

For N entries, each visitor takes O(N) time when a search predicate is O(1). Recursion uses O(H) stack space for depth H; search stores O(K) matching files. Returned result snapshots also require O(K) copying.

6. In-class defense preparation

A live demonstration should show Demo.java, run ./run.sh, and trace root.accept(sizeVisitor) into Directory.accept, SizeCalculatorVisitor.visitDirectory, child.accept(this), and File.accept. Switching to searchVisitor demonstrates that the same element invokes a different concrete operation. Run ./test.sh and show the later counting commit to demonstrate correctness and extension.

The accompanying docs/defense-guide.md includes a step-by-step walkthrough and answers about dispatch, encapsulation, visitor lifecycle, new element types, overflow, complexity, and traversal choices. The live defense requires explaining and running the work in class.

7. Conclusion: encapsulation trade-offs

Visitor successfully separates three operations from a stable two-element hierarchy. The main encapsulation trade-off is structural visibility: visitors must read file size and directory children. Narrow getters provide the needed information without exposing setters or mutable child lists. Copying the constructor's list prevents later caller edits, while immutable result snapshots protect search state.

Read-only access still couples visitors to the public data model. A future change to what a file or directory exposes may require visitor updates. Similarly, introducing a third element type requires a new interface method and corresponding handlers in every visitor. This is the central trade-off between adding operations easily and changing the element hierarchy.

Visitors accumulate state, so a fresh instance is used for every independent operation. The model deliberately represents a small logical hierarchy: it does not read disk contents, handle symlinks, or deduplicate shared objects. Entries are processed once per occurrence. Extremely deep nesting can exhaust the call stack; an iterative traversal would be appropriate if that becomes a real requirement.

8. Public repository and submission

https://github.com/Ziyadin23/visitor-pattern-assignment-6

The repository contains complete source code, 30 automated tests, run scripts, incremental commits, a README, UML diagram sources, captured output, this report, and the defense guide. The student must upload the report and repository link to Moodle and attend the defense. Exact Week 9/10 calendar dates must be checked against the course calendar.

Project materials

Assignment_6_Visitor_Pattern_Instructions.docx: supplied task specification.
Implementation: src/main/java/edu/aitu/visitor/.
Validation: src/test/java/edu/aitu/visitor/VisitorPatternTest.java.
Diagram and output: docs/uml.puml and docs/sample-output.txt.